Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number: 2520030049
Name: S.SARVESWARA REDDY

AOOP

CO-1: TEAM-BASED CASE STUDY EXERCISES

2026

Case Study 5: User Accounts, Email Verification and Contact Details

Case Study:

A Software-as-a-Service company needs user registration, email verification, optional phone numbers, postal addresses, activation, suspension and deletion. The current design permits malformed email addresses, a verified flag without a verification date, expired verification tokens, inconsistent phone formats, incomplete postal addresses, suspended users logging in and duplicate email addresses that differ only in capitalization. The redesigned model uses domain-specific value objects, explicit verification states, controlled account transitions and database-enforced uniqueness.

Q1. Identification of Invariants (6 Marks)

1. **Valid email address:** Every registered email must be non-empty and pass the application email-format validation. **Importance / consequence:** Avoids unusable contact addresses and failed verification delivery.

2. **Verification state consistency:** An email is either pending verification with an active challenge or verified with a verification timestamp; contradictory combinations are not allowed. **Importance / consequence:** Prevents a user being marked verified without proof or a verification date.

3. **Token validity and expiry:** A verification token must match the issued challenge, be unused and be redeemed before its expiry time. **Importance / consequence:** Prevents expired, replayed or guessed links from verifying an account.

4. **Phone number normalization:** When a phone number is supplied, store it in one canonical format, such as E.164. **Importance / consequence:** Avoids duplicate or ambiguous representations of the same number.

5. **Postal address completeness:** If an address is supplied, all fields required by the supported address policy must be present and validated. **Importance / consequence:** Prevents incomplete shipping, billing or regional records.

6. **Account status and login:** Only an eligible ACTIVE account may authenticate; SUSPENDED and DELETED accounts must be denied. **Importance / consequence:** Prevents restricted or removed users from accessing the service.

7. **Email uniqueness:** The canonical email key must be unique across all accounts, regardless of capitalization or surrounding spaces. **Importance / consequence:** Prevents multiple identities from claiming the same mailbox.

8. **Email-change verification:** Changing an already verified email creates a new pending verification for the new address; the new address is not treated as verified until confirmed. **Importance / consequence:** Prevents verification of an old address from being incorrectly transferred to a new one.

Q2. Entities and Value Objects (6 Marks)

User Account - Entity: It has a stable UserIdentifier and a lifecycle (registered, active, suspended, deleted). Its email, contact information and state may change while it remains the same account.

User Identifier - Value Object: It is an immutable identifier value. It has value equality and no independent business lifecycle.

Email Address - Value Object: It is validated and normalized. Two instances with the same canonical representation are equal.

Phone Number - Value Object: It stores one normalized phone representation, such as E.164. It is optional on the account but immutable when represented as a value.

Postal Address - Value Object: It is a collection of validated address values. It has no independent identity and can be replaced as a whole when changed.

Verification Token - Value Object: It is an opaque, immutable token value with an expiry associated with a verification challenge. It is compared by its secret/hash value and expiry, not as a business object. If the system needs an independent audit lifecycle, model the containing VerificationChallenge as an entity with its own ID and consumed/expired state.

The User Account is the entity because its identity persists while its fields change. Contact details and identifiers describe values. A token is best treated as a value carried by a verification challenge; a separately tracked challenge can be an entity if audit requirements need it.

Q3. Making Illegal States Unrepresentable (8 Marks)

The basic class combines independent primitive fields and Boolean flags. This allows invalid combinations such as emailVerified = true with verifiedAt = null, a token with no expiry, both verified and pending data at once, or a suspended user whose login method still succeeds. Nullable strings can also contain whitespace or malformed values.

Recommended domain-specific types

EmailAddress: Validates and stores a canonical email value; normalization is applied before uniqueness checks.

PhoneNumber: Validates and stores a canonical phone number; represented as Optional<PhoneNumber> when not supplied.

PostalAddress: Validates required street, city, postal code and country fields together.

VerificationToken: Contains an opaque token/hash and expiry; it is generated securely and used once.

EmailVerification: A typed state: Pending(token, expiry) OR Verified(verifiedAt), rather than a Boolean plus unrelated nullable fields.

AccountStatus: An explicit status such as ACTIVE, SUSPENDED or DELETED, with login permission checked in one method.

Illustrative Java value types and verification state

```
import
java.time.Instant;
import java.util.*;
```

```
record EmailAddress(String value)
    { private static final Pattern
    BASIC =
        Pattern.compile("^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$");
    EmailAddress {
        if (value == null) throw new IllegalArgumentException("Email
        required"); value = value.trim().toLowerCase(Locale.ROOT); //
        canonical uniqueness key if (!BASIC.matcher(value).matches())
            throw new IllegalArgumentException("Invalid email format");
    }
}

record PhoneNumber(String e164)
    { PhoneNumber {
        if (e164 == null || !e164.matches("\\+[1-9][0-9]{7,14}"))
            throw new IllegalArgumentException("Use a valid E.164 number");
    }
}

record PostalAddress(String street, String city, String postalCode, String country)
    { PostalAddress {
        if (blank(street) || blank(city) || blank(postalCode) ||
            blank(country)) throw new IllegalArgumentException("Address
            fields are required");
    }
    private static boolean blank(String s) { return s == null || s.isBlank(); }
}

record VerificationToken(String tokenHash, Instant expiresAt)
    { VerificationToken {
        Objects.requireNonNull(tokenHash);
        Objects.requireNonNull(expiresAt);
    }
    boolean isExpired(Instant now) { return !now.isBefore(expiresAt); }
}

sealed interface EmailVerification permits PendingEmail, VerifiedEmail
{ } record PendingEmail(VerificationToken token) implements
EmailVerification {
    PendingEmail { Objects.requireNonNull(token); }
}
```

The email expression shown is a basic application-level format check, not a complete implementation of every email standard. A production service should use a well-tested parser and still verify ownership by sending a challenge. The canonicalization rule (including lower-casing) is an explicit product policy used to meet the assignment requirement that capitalization differences do not create separate accounts.

Account representation and validation behaviour

```
final class UserAccount
    { private final UUID id;
    private EmailAddress
```

```
private Optional<PhoneNumber> phone;
private Optional<PostalAddress>
address; private AccountStatus status;

boolean canLogin() {
    return status == AccountStatus.ACTIVE
        && verification instanceof VerifiedEmail;
}
void suspend() {
    if (status == AccountStatus.DELETED) throw new
    IllegalStateException(); status = AccountStatus.SUSPENDED;
}
void activate() {
    if (status != AccountStatus.SUSPENDED) throw new
    IllegalStateException(); status = AccountStatus.ACTIVE;
}
void delete() { status = AccountStatus.DELETED; }
// No public setters for verification state or account status.
```

A registration policy may decide whether the account status becomes ACTIVE only after verification. Regardless of that policy, canLogin must enforce both the required account status and verified email state. Optional phone and address data should use Optional types at the API boundary and be validated before being stored.

Q4. Email Verification and Account Lifecycle (6 Marks)

1. **Registration information:** Maintain a generated UserIdentifier, canonical EmailAddress, registration timestamp, pending verification challenge with expiry, account status, and any optional validated phone/address. Do not mark the email verified during registration.

2. **Token generation and validation:** Generate a cryptographically random, high-entropy token (for example, 32 random bytes encoded with URL-safe Base64). Send the raw token to the user once, but store only a secure hash of it with the expiry and challenge ID. On redemption, hash the submitted token and compare it safely with the stored hash; also check the user, challenge status and expiry.

3. **Expired token:** Reject it and leave the email pending. Provide a resend action that creates a new token/challenge, invalidates the previous one and sets a new expiry. Rate-limit resends and verification attempts.

4. **Verified user changes email:** Normalize and check uniqueness of the new email. Store it as a pending new email or pending change, issue a fresh verification token and require confirmation. Do not transfer the old address verification to the new address. The product may retain the old verified address until the new one is confirmed, or restrict sensitive actions during the change; the chosen policy must be explicit.

5. **Suspended/deleted login:** The authentication service calls canLogin (or an equivalent account policy) on every login and session refresh. SUSPENDED and DELETED must return a denial. Suspension should revoke or invalidate existing sessions; deletion should follow the system retention policy and also prevent new authentication.

6. **Preventing contradictory states:** Replace emailVerified, verifiedAt, verificationToken and tokenExpiry as independent fields with one EmailVerification state. PendingEmail contains a valid challenge; VerifiedEmail contains a non-null timestamp. Transition methods replace one state with the other and remove the pending token when verification succeeds.

Registration and verification workflow

1. Receive registration request and construct validated EmailAddress and optional contact value objects.
2. Check canonical email uniqueness, then insert the user and pending challenge in one transaction protected by a database unique constraint.
3. Generate a secure random token, store its hash and expiry, and send the raw token through the email service.
4. When the link is opened, validate token hash, challenge status and expiry; reject expired, used or mismatched tokens.
5. On success, mark the challenge consumed and replace PendingEmail with VerifiedEmail(verifiedAt) in one transaction.
6. On email change, repeat the process for the new normalized address; on suspension/deletion, block login and revoke sessions.

Q5. Cross-Aggregate Rules and Design Evaluation (4 Marks)

1. **Is uniqueness an invariant of one User object?** No. A single UserAccount can validate that its own EmailAddress is well-formed, but it cannot know whether a different account already uses that address. Uniqueness is a rule across the user collection/repository, so it is a cross-aggregate invariant.

2. **Enforcement:** Normalize the email first, then have a registration/domain service ask the user repository whether the canonical key is already used. The check improves the user experience, but it is not sufficient by itself under concurrency.

3. **Simultaneous registration:** Two requests can both check and see no account, then both attempt to insert. Without a database constraint, duplicate accounts may be created.

4. **Database protection:** Create a UNIQUE constraint or unique index on the canonical/normalized email column. Insert in a transaction and handle the unique-constraint violation as “email already registered”. This makes the database the final authority even when requests race.

5. **Strong typing - advantage and disadvantage:** Advantage: invalid combinations become harder to construct and validation is centralized in the type. Disadvantage: more classes and conversion code are required, and developers must learn the model. The improved safety is worth the extra structure for security-sensitive account and verification rules.

Common team submission and presentation checklist

- Problem analysis: explain malformed contact data, contradictory verification fields, expired tokens, suspended login and duplicate emails.
- Invariant identification: present at least seven rules and the consequences of violating them.

- Domain classification: justify UserAccount, UserIdentifier, EmailAddress, PhoneNumber, PostalAddress and VerificationToken.
- UML class diagram: show UserAccount, EmailAddress, EmailVerification, PendingEmail, VerifiedEmail, VerificationToken and optional contact values.
- Java implementation: demonstrate constructor validation, token expiry handling and canLogin checks.
- Design justification: explain strong types, canonicalization, secure one-time tokens and database uniqueness.
- Team presentation: demonstrate one successful verification, one expired token rejection and one duplicate-email registration rejection.